



254383 Algorithm Design and Analysis

อ.พิเศษพงศ์ สุธาพันธ์

ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ





Week 7

Space and Time Trade-Offs

อ.พิเศษพงศ์ สุธาพันธ์

ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ





Space and Time Trade-Offs

วัตถุประสงค์

1. รู้จักเทคนิค Space and Time Trade-offs แบบต่าง
2. สามารถนำไปประยุกต์ใช้งานได้



Space and Time Trade-Offs

แนวคิดแบบ input enhancement

- Counting methods for sorting
- Boyer-Moore algorithm for string matching
- Horspool algorithm



Space and Time Trade-Offs

แนวคิดแบบ prestructuring

- Hashing
- Indexing with B-trees



Space and Time Trade-Offs

7.1 Sorting by Counting



Space and Time Trade-Offs

Sorting by Counting หรือ Comparison counting sort

Array $A[0..5]$

62	31	84	96	19	47
----	----	----	----	----	----

Initially

Count []

0	0	0	0	0	0
---	---	---	---	---	---

After pass $i = 0$

Count []

3	0	1	1	0	0
---	---	---	---	---	---

After pass $i = 1$

Count []

	1	2	2	0	1
--	---	---	---	---	---

After pass $i = 2$

Count []

		4	3	0	1
--	--	---	---	---	---

After pass $i = 3$

Count []

			5	0	1
--	--	--	---	---	---

After pass $i = 4$

Count []

				0	2
--	--	--	--	---	---

Final state

Count []

3	1	4	5	0	2
---	---	---	---	---	---

Array $S[0..5]$

19	31	47	62	84	96
----	----	----	----	----	----



Space and Time Trade-Offs

Sorting by Counting หรือ Comparison counting sort

ALGORITHM *ComparisonCountingSort*($A[0..n - 1]$)

//Sorts an array by comparison counting

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $S[0..n - 1]$ of A 's elements sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 1$ **do** $Count[i] \leftarrow 0$

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[i] < A[j]$

$Count[j] \leftarrow Count[j] + 1$

else $Count[i] \leftarrow Count[i] + 1$

for $i \leftarrow 0$ **to** $n - 1$ **do** $S[Count[i]] \leftarrow A[i]$

return S



Space and Time Trade-Offs

ประสิทธิภาพ

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

$$= \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1]$$

$$= \sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}$$



Space and Time Trade-Offs

Distribution counting

13	11	12	13	12	12
----	----	----	----	----	----

Array values	11	12	13
Frequencies	1	3	2
Distribution values	1	4	6



Space and Time Trade-Offs

Distribution counting

$A[5] = 12$

$A[4] = 12$

$A[3] = 13$

$A[2] = 12$

$A[1] = 11$

$A[0] = 13$

$D[0..2]$

1	4	6
1	3	6
1	2	6
1	2	5
1	1	5
0	1	5

$S[0..5]$

			12		
		12			
					13
	12				
11					
				13	



Space and Time Trade-Offs

ALGORITHM *DistributionCountingSort*($A[0..n-1]$, l , u)

//Sorts an array of integers from a limited range by distribution counting

//Input: An array $A[0..n-1]$ of integers between l and u ($l \leq u$)

//Output: Array $S[0..n-1]$ of A 's elements sorted in nondecreasing order

for $j \leftarrow 0$ **to** $u - l$ **do** $D[j] \leftarrow 0$ //initialize frequencies

for $i \leftarrow 0$ **to** $n - 1$ **do** $D[A[i] - l] \leftarrow D[A[i] - l] + 1$ //compute frequencies

for $j \leftarrow 1$ **to** $u - l$ **do** $D[j] \leftarrow D[j - 1] + D[j]$ //reuse for distribution

for $i \leftarrow n - 1$ **downto** 0 **do**

$j \leftarrow A[i] - l$

$S[D[j] - 1] \leftarrow A[i]$

$D[j] \leftarrow D[j] - 1$

return S



Space and Time Trade-Offs

7.2 Input Enhancement in String Matching



Space and Time Trade-Offs

Horspool's algorithm

s_0 ... s_{n-1}
 B A R B E R

กำหนด text เป็น s_0 ถึง s_{n-1}

Pattern คือ BARBER



มีเหตุการณ์เกิดขึ้นได้ 4 กรณี คือ

$$s_0 \quad \dots \quad S \quad \dots \quad s_{n-1}$$

$$\quad \quad \quad \parallel$$

$$\quad \quad \quad \text{B A R B E R} \rightarrow \text{B A R B E R}$$

fppt.com



กรณีศึกษาที่ 2

$s_0 \quad \dots \quad B \quad \dots \quad s_{n-1}$

$\parallel$

B A B A R B E R



ตัวสุดท้ายเปรียบเทียบกับแล้วไม่เหมือน แต่ไปเหมือนกับ
ตัวอื่นใน pattern ให้เลื่อนไปจนตัวที่เหมือนไปตรงกัน



Space and Time Trade-Offs

Horspool's algorithm

กรณีที่ 3

s_0 ... M E R ... s_{n-1}
 X || ||
 L E A D E R → L E A D E R

จากตัวอักษรท้ายสุดมาถึงหลังตัวไม่เหมือน ถ้าไม่มีใน pattern ให้เลื่อนไป หลังตำแหน่งที่เปรียบเทียบเลย



Space and Time Trade-Offs

Horspool's algorithm

กรณีี่ที่ 4

$s_0 \quad \dots \quad A \quad R \quad \dots \quad s_{n-1}$
 $\quad \quad \quad \backslash \quad \parallel$
 $\quad \quad \quad \text{R E O R E O R D E R}$
 $\quad \quad \quad \text{--- R E O \rightarrow}$

จากตัวอักษรท้ายสุดมาถึงหลังตัวไม่เหมือน ถ้ายังมีใน pattern ให้เลื่อนไปยังตำแหน่งที่เหมือน



Space and Time Trade-Offs

Horspool's algorithm

จาก pattern สามารถนำไปสร้างตารางสำหรับการเลื่อน

ALGORITHM *ShiftTable*($P[0..m - 1]$)

//Fills the shift table used by Horspool's and Boyer-Moore algorithms

//Input: Pattern $P[0..m - 1]$ and an alphabet of possible characters

//Output: $Table[0..size - 1]$ indexed by the alphabet's characters and

// filled with shift sizes computed by formula (7.1)

for $i \leftarrow 0$ **to** $size - 1$ **do** $Table[i] \leftarrow m$

for $j \leftarrow 0$ **to** $m - 2$ **do** $Table[P[j]] \leftarrow m - 1 - j$

return $Table$



Space and Time Trade-Offs

ALGORITHM *HorspoolMatching*($P[0..m-1]$, $T[0..n-1]$)

//Implements Horspool's algorithm for string matching

//Input: Pattern $P[0..m-1]$ and text $T[0..n-1]$

//Output: The index of the left end of the first matching substring

// or -1 if there are no matches

ShiftTable($P[0..m-1]$) //generate *Table* of shifts

$i \leftarrow m-1$ //position of the pattern's right end

while $i \leq n-1$ **do**

$k \leftarrow 0$ //number of matched characters

while $k \leq m-1$ **and** $P[m-1-k] = T[i-k]$ **do**

$k \leftarrow k+1$

if $k = m$

return $i-m+1$

else $i \leftarrow i + \text{Table}[T[i]]$

return -1



Space and Time Trade-Offs

ตัวอย่าง การสร้างตาราง

character c	A	B	C	D	E	F	...	R	...	Z	_
shift $t(c)$	4	2	6	6	1	6	6	3	6	6	6



B A R B E R
4 3 2 1 0



Space and Time Trade-Offs

ตัวอย่าง

character c	A	B	C	D	E	F	...	R	...	Z	_
shift $t(c)$	4	2	6	6	1	6	6	3	6	6	6

J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
B A R B B B A R B E R B A B A R B A R B E R



Space and Time Trade-Offs

Boyer-Moore Algorithm

s_0	...	c	s_{i-k+1}	...	s_i	...	s_{n-1}	text
		$\nparallel$	$\parallel$		$\parallel$			
p_0	...	p_{m-k-1}	p_{m-k}	...	p_{m-1}			pattern

จะมีการ shift แบบ bad symbol shift

โดย $t_1(c) - k$

k คือ จำนวนตัวอักษรที่เหมือน

c คือ ตัวอักษรที่ไม่เหมือน



Space and Time Trade-Offs

Boyer-Moore Algorithm

ตัวอย่าง

character c	A	B	C	D	E	F	...	R	...	Z	_
shift $t(c)$	4	2	6	6	1	6	6	3	6	6	6

s_0 ... S E R ... s_{n-1}
 X || ||
 B A R B E R
 B A R B E R

$$t_1(S) - 2 = 6 - 2 = 4$$



Space and Time Trade-Offs

Boyer-Moore Algorithm

ตัวอย่าง

character c	A	B	C	D	E	F	...	R	...	Z	_
shift $t(c)$	4	2	6	6	1	6	6	3	6	6	6

$s_0 \quad \dots \quad A \quad E \quad R \quad \dots \quad s_{n-1}$

$\quad \quad \quad \times \quad || \quad ||$

$B \quad A \quad R \quad B \quad E \quad R$

$\quad \quad B \quad A \quad R \quad B \quad E \quad R$

$$t_1(A) - 2 = 4 - 2 = 2$$



Space and Time Trade-Offs

Boyer-Moore Algorithm

แต่ในกรณี

$$t_1(c) - k \leq 0$$

จะใช้

$$d_1 = \max\{t_1(c) - k, 1\}.$$



Space and Time Trade-Offs

Boyer-Moore Algorithm

และมีการ shift แบบที่ 2 คือ good-suffix shift

k	pattern	d_2
1	ABC <u>B</u> AB	2
2	ABC <u>B</u> AB	4
3	ABC <u>B</u> AB	4
4	ABC <u>B</u> AB	4
5	ABC <u>B</u> AB	4



Space and Time Trade-Offs

Boyer-Moore Algorithm

แต่เวลา **shift** จะเลือกเอาค่าดังนี้

$$d = \begin{cases} d_1 & \text{if } k = 0, \\ \max\{d_1, d_2\} & \text{if } k > 0, \end{cases}$$



Space and Time Trade-Offs

Boyer-Moore Algorithm

ตัวอย่าง

ง

c	A	B	C	D	...	0	...	Z	_
$t_1(c)$	1	2	6	6	6	3	6	6	6

bad-symbol table

k	pattern	d_2
1	BAO <u>B</u> AB	2
2	BAO <u>B</u> AB	5
3	BAO <u>B</u> AB	5
4	BAO <u>B</u> AB	5
5	BAO <u>B</u> AB	5

good-suffix table



Space and Time Trade-Offs

Boyer-Moore Algorithm

ตัวอย่าง

B E S S _ K N E W _ A B O U T _ B A O B A B S

B A O B A B B A O B A B A O B A B A O B A B

$$d_1 = t_1(K) - 0 = 6$$

$$d_1 = t_1(_) - 2 = 4$$

$$d_1 = t_1(_) - 1 = 5$$

$$d_2 = 5$$

$$d_2 = 2$$

$$d = \max\{4, 5\} = 5$$

$$d = \max\{5, 2\} = 5$$



Space and Time Trade-Offs

Boyer-Moore Algorithm

แบบฝึกหัด

GCATCGCAGAGAGTATACAGTACG

TATACA



Space and Time Trade-Offs

7.3 Hashing



Space and Time Trade-Offs

Hashing

ประสิทธิภาพคือ $\Theta(1)$

แนวคิด คือ มีการแปลงข้อของข้อมูลไปเป็นที่อยู่

keys	A	FOOL	AND	HIS	MONEY	ARE	PARTED
hash addresses	1	9	6	10	7	11	12

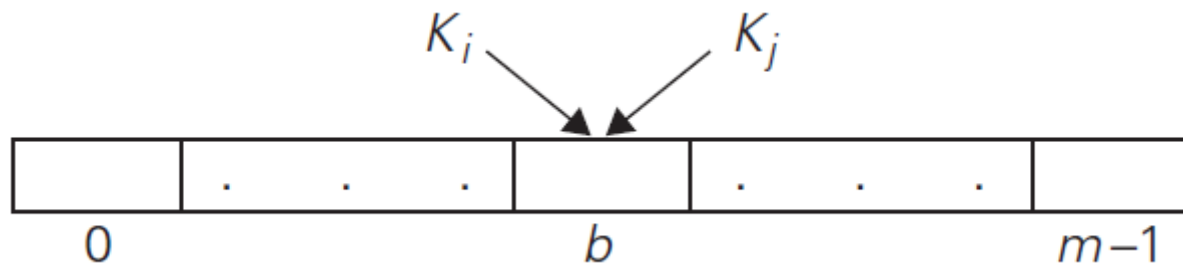
0	1	2	3	4	5	6	7	8	9	10	11	12
	A					AND	MONEY		FOOL	HIS	ARE	PARTED



Space and Time Trade-Offs

Hashing

แต่ในกรณีที่แปลงข้อมูลเป็นที่อยู่แล้วที่อยู่ชนกัน



มีวิธีแก้ 2 แบบ คือ

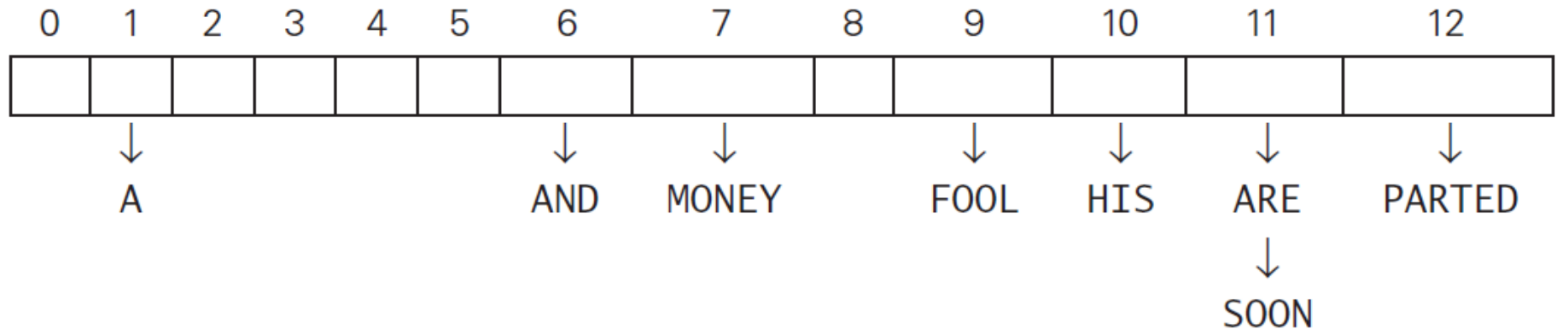
1. Open Hashing (Separate Chaining)
2. Closed Hashing (Open Addressing)



Space and Time Trade-Offs

Open Hashing (Separate Chaining)

keys	A	FOOL	AND	HIS	MONEY	ARE	SOON	PARTED
hash addresses	1	9	6	10	7	11	11	12





Space and Time Trade-Offs

Closed Hashing (Open Addressing)

keys	A	FOOL	AND	HIS	MONEY	ARE	SOON	PARTED
hash addresses	1	9	6	10	7	11	11	12

0	1	2	3	4	5	6	7	8	9	10	11	12
PARTED	A					AND	MONEY		FOOL	HIS	ARE	SOON



Space and Time Trade-Offs

Closed Hashing (Open Addressing)

keys	A	FOOL	AND	HIS	MONEY	ARE	SOON	PARTED
hash addresses	1	9	6	10	7	11	11	12

0	1	2	3	4	5	6	7	8	9	10	11	12
PARTED	A					AND	MONEY		FOOL	HIS	ARE	SOON



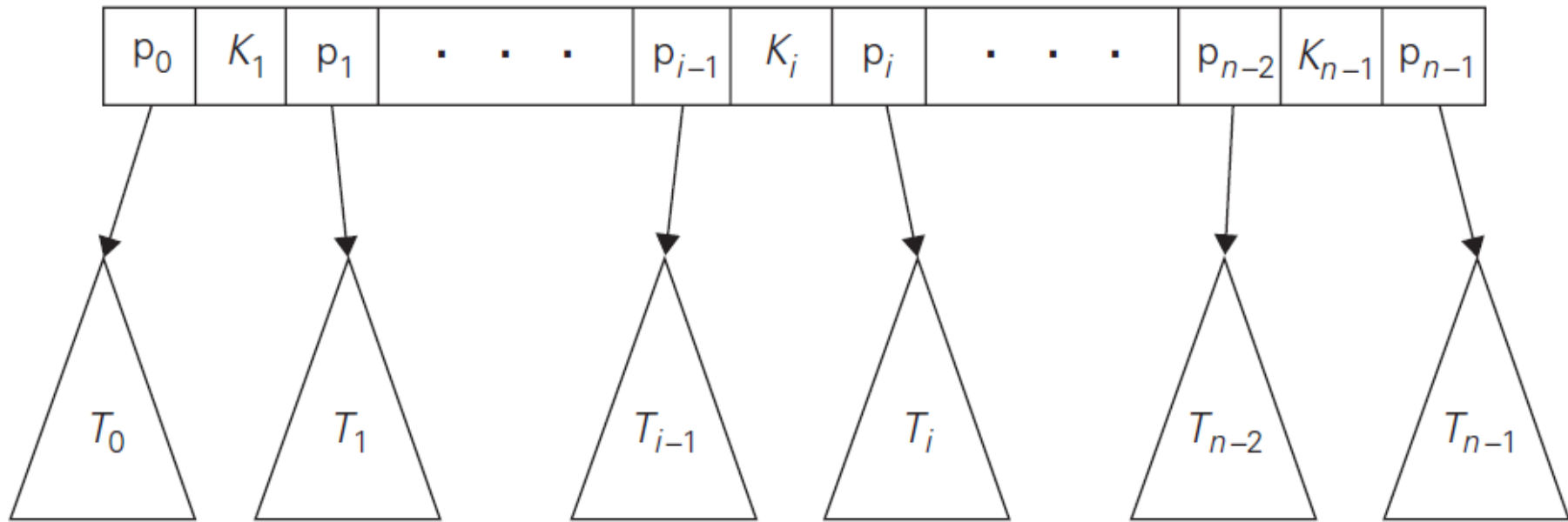
Space and Time Trade-Offs

7.4 B-Trees



Space and Time Trade-Offs

B-Trees

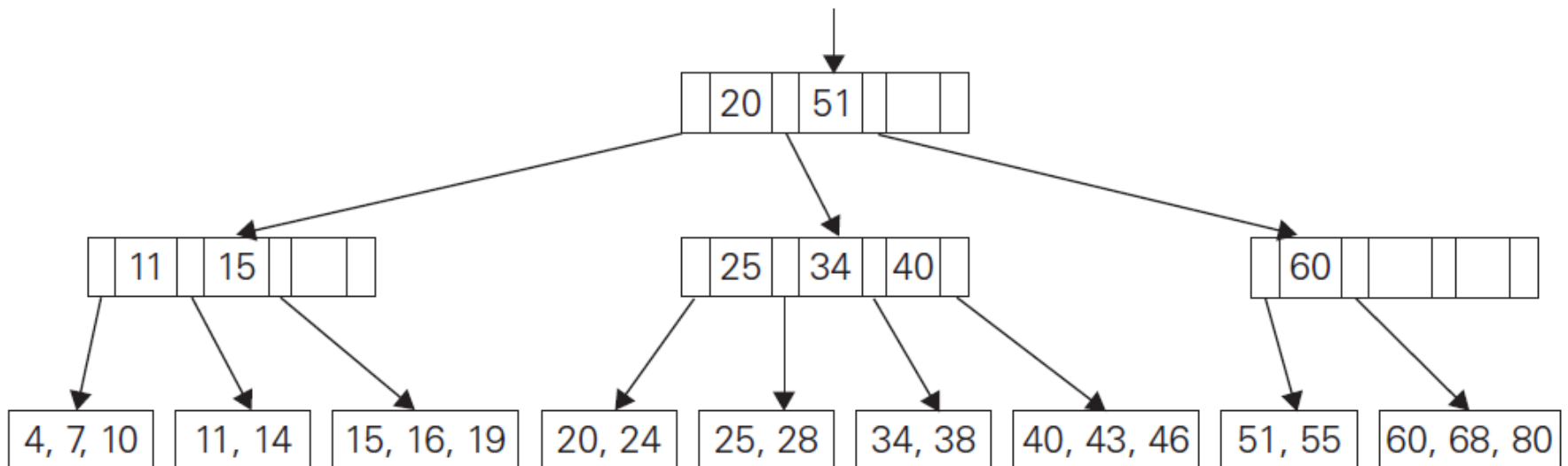


Parental node of a B-tree



Space and Time Trade-Offs

B-Trees

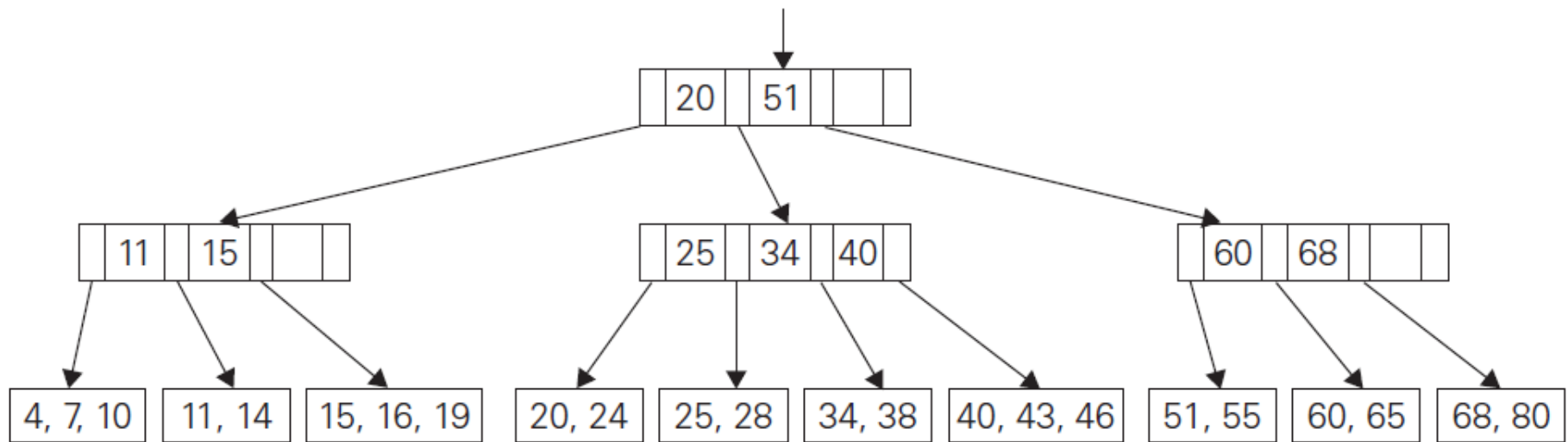


Example of a B-tree of order 4



Space and Time Trade-Offs

B-Trees



B-tree obtained after inserting 65 into the B-tree



อ้างอิงและต้นฉบับ

[1] Introduction to the design & analysis of algorithm, 3rd,
Anany Levitin



Space and Time Trade-Offs

สรุป(ทบทวน)

1. Sorting by Counting เป็นการ
จัดเรียงโดยการนับจำนวนตัวที่น้อย
กว่าแล้วเอาจำนวนที่นับได้นั้นไปเป็น
ตำแหน่งในอาร์เรย์ที่จะเอาค่า
นั้นลงไปแทน



Space and Time Trade-Offs

สรุป(บททวน)

เช่น

10	5	0	8	15	20
----	---	---	---	----	----

นำจำนวนที่น้อยกว่า

3	1	0	2	4	5
---	---	---	---	---	---

0 1 2 3 4 5

--	--	--	--	--	--



Space and Time Trade-Offs

สรุป(บททวน)

2. Distribution Counting เป็นการ
จัดเรียงโดยมีการนับจำนวน 2 อย่าง
คือ นับจำนวนซ้ำ และ นับตัวที่น้อย
หรือเท่ากับ โดยจะเอาจำนวนที่นับได้
นั้นไปเป็นค่าพจน์เป็นตำแหน่งอื่น
เติกซ์ในอาร์เรย์ที่จะเอาค่านั้นลงไป



Space and Time Trade-Offs

สรุป(บททวน)

i->	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
a[i]->	3	1	1	0	3	7	5	5	2	4	2	1	0	2	6	4

count[i]	0	1	2	3	4	5	6	7
	2	3	3	2	2	2	1	1

count[i]	0	1	2	3	4	5	6	7
	2	5	8	10	12	14	15	16

b[i]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	0	0	1	1	1	2	2	2	3	3	4	4	5	5	6	7



Space and Time Trade-Offs

สรุป(ทบทวน)

3. Horspool's algorithm เป็นการ
หาสตริงจากบทความ โดยมีการสร้าง
ตารางสำหรับใช้ shift



Space and Time Trade-Offs

สรุป(ทบทวน)

4. Boyer-Moore algorithm เป็นการหาสตริงจากบทความ โดยมีการสร้างตารางสำหรับใช้ shift 2 ตาราง คือ a) **Bad Character Rule** และ b) **Good Suffix Rule**



Space and Time Trade-Offs

สรุป(ทบทวน)

**5. Hashing เป็นการทำให้ค่าของ
ข้อมูลไปเป็นตำแหน่งในหน่วยความจำ**



Space and Time Trade-Offs

สรุป(ทบทวน)

6. B-tree เป็นการเก็บข้อมูลในรูป
ต้นไม้ แต่ละโหนดสามารถเก็บได้หลาย
ค่า และ โหนดลูกจะมีด้านซ้ายจะมีค่า
น้อยกว่าค่าคีย์ และโหนดลูกด้านขวา
จะมีค่ามากกว่าค่าคีย์เสมอ โดย
ส่วนมากจะใช้ในการทำ index ใน

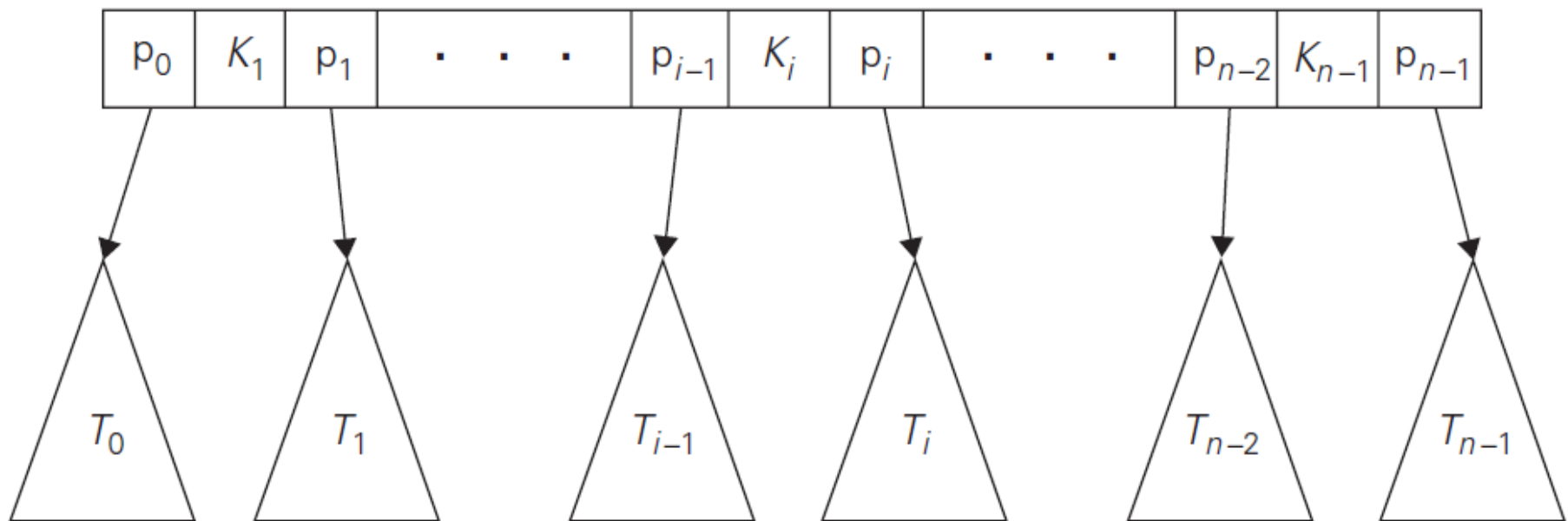
ฐานข้อมูล



Space and Time Trade-Offs

สรุป(บททวน)

ตัวอย่าง





Space and Time Trade-Offs

สรุป(ทบทวน)

ตัวอย่างการใช้งาน เช่น สมุดโทรศัพท์
ดิกชันนารี